Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки і комп’ютерних технологій

Звіт
про виконання лабораторної роботи №13
з курсу «Сховища знань та управління знаннями»
на тему:
«**Побудова архітектури гібридного сховища знань**»

Виконав:
студент групи ФеІ-42
Трофимов Володимир
Перевірив:
ас. Керод Т. І.

Львів – 2026

Тема

Побудова архітектури гібридного сховища знань (HKB) для предметної області «Цифрові моделі транспортних систем».

Мета

Спроєктувати та задокументувати комплексну архітектуру гібридного сховища знань (Hybrid Knowledge Base, HKB), що поєднує реляційне/колоночне зберігання часових вимірів, графове представлення топології та онтологічних зв'язків (RDF/OWL) і векторне індексування неструктурованих регуляторних регламентів; визначити конвеєри наповнення (ETL/ELT), узгодження з розробленою онтологією, механізми API-доступу та політики якості й безпеки.

Завдання

- провести системну інвентаризацію первинних джерел даних цифрової транспортної системи (структуровані таблиці, неструктурована документація, часові логи, телематика) із зазначенням власників, періодичності оновлення та ризиків якості.
- розробити логічну модель гібридного сховища: розподілити потоки між реляційним, графовим та векторним просторами зберігання.
- спроєктувати фізичну архітектуру HKB (архітектурна діаграма), визначивши стек технологій зберігання, шар інтеграції даних, шину передачі повідомлень та API-фасад.
- забезпечити онтологічний шар: використати формалізовану OWL-онтологію предметної області (≥ 10 класів, ≥ 8 властивостей), зв'язати екземпляри ABox із фізичними записами та сформувати 2–3 правила якості у стандарті SHACL.
- описати схеми завантаження даних (Batch/Stream), стратегію версіонування онтології та інфраструктури (SCD Type 2), а також політики безпеки й розмежування прав доступу (RBAC).
- реалізувати мінімальний діючий прототип гібридного сховища та навести верифіковані приклади виконання аналітичних SQL, SPARQL/Cypher, векторних та комбінованих мультимодальних запитів.

Хід роботи

1. Інвентаризація джерел даних цифрової транспортної системи

Сучасна цифрова модель транспортної системи міста (Digital Transport Model) оперує гетерогенними даними, які суттєво відрізняються структурою, швидкістю генерації (velocity), обсягом та вимогами до затримки обробки (latency). Для проектування цілісної HKB проведено класифікацію та аудит ключових інформаційних потоків муніципальної транспортної екосистеми.

У результаті обстеження виділено 4 основні категорії даних:

- *Високочастотні часові ряди та телематика (Structured Time-Series):* показники швидкості руху, просторові GPS-треки рухомого складу, поточні значення інтенсивності потоків із дорожніх контролерів.
- *Паспорт інфраструктури та топологія мережі (Semi-structured / Graph Data):* цифрова картографія вулично-дорожньої мережі, зв'язність дорожніх сегментів, схеми руху та перелік зупинок громадського транспорту.
- *Неструктуровані регламенти та правила (Unstructured Text / Knowledge Rules):* описи інцидентів, нормативно-правові накази про організацію об'їздів, посадові інструкції диспетчерів, формалізовані умови активації правил прийняття рішень.
- *Транзакційні журнали диспетчеризації (Relational Logs):* журнали зафіксованих нештатних ситуацій (ДТП, дорожні роботи), історія ухвалених коригуючих рішень, розклад руху транспорту.

Зведена карта інвентаризації джерел наведена в Таблиці 1:

ID дже-рела	**Назва джерела та сутності онтології**	**Формат та тип даних**	**Джерело / Власник**	**Періодичність оновлення (SLA)**	**Ризики якості даних**	**Метод нівелювання ризиків**
SRC-01	Бортові GPS-трекери (GPSMeasu-rement, Vehicle)	JSON / Protobuf (Stream)	Муніципальні перевізники, GPS-провайдери	Кожні 5–15 секунд (Real-time)	Втрата сигналу в тунелях/між будівлями, аномальні викиди координат	Фільтрація Калмана, Dead Reckoning, перевірка граничної швидкості
SRC-02	Телематичні датчики швидкості та інтенсивності (TrafficSensor, SpeedData)	Числові матриці, CSV/REST	Центр організації дорожнього руху (ЦОДР)	Кожні 1–5 хвилин (Micro-batch)	Апаратні збої детекторів, «нульові» показники вночі	Заповнення пропусків ковзним середнім, валідація діапазонів (0–120 км/год)
SRC-03	Граф вулично-дорожньої мережі (RoadNetwork, RoadSegment, Intersection)	GeoJSON, Shapefiles, RDF/Graph	Департамент інфраструк-тури та містобуду-вання	За запитом / планово (раз на квартал)	Неузгодженість координат стиків, розриви топології графів	Геометрична валідація поліліній, перевірка цілісності зв'язності (Degree check)
SRC-04	Маршрутна мережа та розклад (Route, BusStop)	GTFS, реляційні таблиці (RDBMS)	Управління транспорту, диспетчер-ська служба	Щомісяця або при сезонних змінах	Застарілі номери рейсів, неузгоджені назви зупинок	Словникова нормалізація, посилальна цілісність зовнішніх ключів (Foreign Keys)
SRC-05	Реєстр транспортних пригод та подій (TrafficAcci-	REST JSON, логи інцидентів	Патрульна поліція, Служба	Подійна поява (Event-driven, по мірі фіксації)	Затримка ручного внесення оператором (до	Обов'язкові поля форми, штамп часу виявлення

ID дже-рела	Назва джерела та сутності онтології	Формат та тип даних	Джерело / Власник	Періодичність оновлення (SLA)	Ризики якості даних	Метод нівелювання ризиків
	dent,RoadWorks)		відновлення доріг		30 хв), неповний опис	vs штамп внесення
SRC-06	Регламенти диспетчерсь-кого регулювання (DecisionRule, TransportDecision)	Документи PDF, текстові інструкції (MD/Docx)	Муніципа-льний диспетчер-ський центр	При зміні регуляторної бази (рідко)	Неоднозначність формулювань у природній мові, суперечливі правила	Семантичне векторне індексування, валідація через правила логічного виведення

2. Розробка логічної моделі гібридного сховища знань (HKB)

Гібридне сховище знань проєктується відповідно до принципу поліглотного зберігання (Polyglot Persistence): кожен тип даних розміщується в тій моделі сховища, яка забезпечує найвищу продуктивність вибірок та природне представлення сутностей.

Логічний простір HKB декомпоновано на три взаємодоповнюючі рівні: Реляційний/Колоночний, Графовий та Векторний.

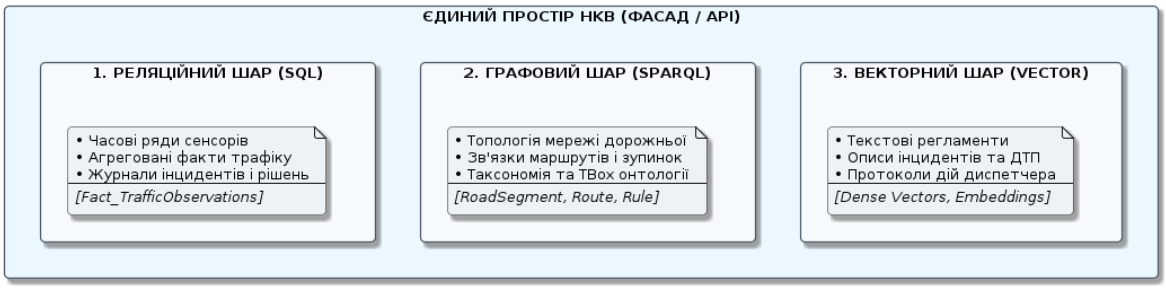


Рис.1 – Логічний простір HKB

2.1. Реляційний / Колоночний шар (Structured & Time-Series Data)

Призначення: Зберігання структурованих параметричних даних з високою інтенсивністю запису (часові ряди телематики), фіксація фактів моніторингу та агрегованих показників для швидкого виконання аналітичних SQL-запитів і підрахунку статистик.

Модель даних: Багатовимірна схема або нормалізовані таблиці часових рядів із сурогатними та бізнес-ключами.

Ключові сутності:

- *fact_traffic_observations* — таблиця фактів періодичних вимірювань.
  Поля: ObservationID (PK), Timestamp / DateKey, SegmentID (FK), RouteID (FK), VehicleCount (інтенсивність, авто/год), ObservedSpeed (середня швидкість, км/год), DelayMinutes (затримка, хв), CongestionLevel (рівень затору).

- *dim_road_segment* — таблиця паспорта дорожніх ділянок.
  Поля: SegmentID (PK, бізнес-код: DS-001), SegmentName, District, RoadType, LanesCount, AllowedSpeed, LengthMeters.
- *dim_incident_log* — журнал оперативного обліку подій.
  Поля: IncidentID (PK: ACC-001), SegmentID (FK), EventType (Accident, RoadWorks, RoadClosure), SeverityLevel (minor, moderate, major), ReportedAt (DateTime).

2.2. Графовий шар (Graph / RDF / Ontology Alignment)

Призначення: Представлення топологічної зв'язності вулично-дорожньої мережі, просторово-логічних ланцюгів громадського транспорту, причинно-наслідкових зав'язків між інцидентами й диспетчерськими рішеннями, а також формальне виведення знань (reasoning).

Узгодження з онтологією (Ontology Alignment): Графовий шар безпосередньо відображає концептуальну модель update.owl. Кожен об'єкт отримує унікальний глобальний ідентифікатор (URI):

- *Класи онтології (TBox) → Типи/Мітки вершин:* :RoadSegment, :Intersection, :BusStop, :Route, :Vehicle, :TrafficAccident, :DecisionRule, :TransportDecision.
- *Об'єктні властивості → Напрямлені ребра (Edges / Predicates):*
  - (:RoadSegment)-[:NETWORK {travelTimeMin}]->(:RoadSegment) — зв'язність топології;
  - (:BusStop)-[:LOCATED_ON]->(:RoadSegment) — просторова прив'язка зупинок;
  - (:Route)-[:COVERS_SEGMENT]->(:RoadSegment) та [:SERVES_STOP]->(:BusStop) — маршрутна мережа;
  - (:TrafficAccident)-[:OCCURRED_ON]->(:RoadSegment) — локалізація інциденту;
  - (:TrafficAccident)-[:AFFECTS]->(:TrafficFlow) — вплив на потік;
  - (:DecisionRule)-[:DETERMINES]->(:TransportDecision) — зв'язок регуляторних правил із рішеннями;
  - (:TransportDecision)-[:CHANGES]->(:Route) — зміна схеми руху автобусів.
- *Спільні ключі (Federation Anchors):* Рядок реляційної таблиці зі значенням SegmentID = 'DS-001' однозначно зв'язується з графовим вузлом http://example.org/transport-ontology#DS001 (або :RoadSegment {id: 'DS-001'}).

2.3. Векторний шар (Vector Indexing & Semantic Search)

Призначення: Зберігання щільних числових векторів (ембеддингів) для неструктурованих текстів природною мовою: посадових інструкцій диспетчера, текстових регламентів дій при заторах, описів інцидентів та формулювань

експертних правил прийняття рішень. Це дозволяє виконувати семантичний пошук регламентів за неточним запитом (Semantic / Dense Retrieval) і реалізувати архітектуру RAG (Retrieval-Augmented Generation).

Специфікація векторизації:

- *Модель ембеддингів:* sentence-transformers/all-MiniLM-L6-v2 (або BAAI/bge-m3).
- *Розмірність векторного простору (D):* 384 виміри (для MiniLM) або 1024 виміри (для bge-m3).
- *Метрика подібності (Distance Metric):* Косинусна подібність (Cosine Similarity) / Косинусна відстань.
- *Індексовані поля та метадані.*

Специфікація об'єктів векторного індексування НКВ:

Об'єкт індексування	**Вхідне текстове поле (Chunk / Payload)**	**Зв'язок з онтологією**	**Метадані (Payload filter)**
Регламенти рішень (Decision Rules)	description: Текстове формулювання умов активації та заходів (наприклад: *"Якщо рівень завантаженості дороги високий, необхідно змінити маршрут рухомого складу та сповістити пасажирів"*).	DecisionRule (RuleHighCongestion, RuleLowSpeed)	rule_id, category: "rerouting", severity_threshold: 0.75
Протоколи ДТП (Accident Reports)	description: Детальний опис нештатної ситуації, зафіксованої патрулем чи свідками (наприклад: *"ДТП за участі вантажівки, перекрито дві смуги руху в бік центру, ускладнено проїзд громадського транспорту"*).	TrafficAccident (Accident001)	incident_id, segment_id, severity: "major", injured: true
Регламенти реагування (SOP Manuals)	Повний фрагмент інструкції чергового диспетчера ЦОДР щодо алгоритму перерозподілу світлофорних фаз та виклику дорожніх служб.	TransportDecision (AlternativeRouteDecision, TrafficRegulation)	doc_id, target_role: "dispatcher", action_type: "signal_control"

3. Фізична архітектура гібридного сховища знань

Фізична архітектура розроблена за багаторівневим сервіс-орієнтованим принципом для забезпечення високої пропускної здатності, відмовостійкості, ізольованого масштабування компонентів і підтримки різнорідних запитів (мультимодального доступу).

3.1. Архітектурна схема НКВ (Hybrid Knowledge Base Architecture)

Архітектура об'єднує 5 функціональних шарів: збору первинних потоків, збереження даних, ETL/семантичної інтеграції, уніфікованого доступу та сервісів моніторингу/безпеки.

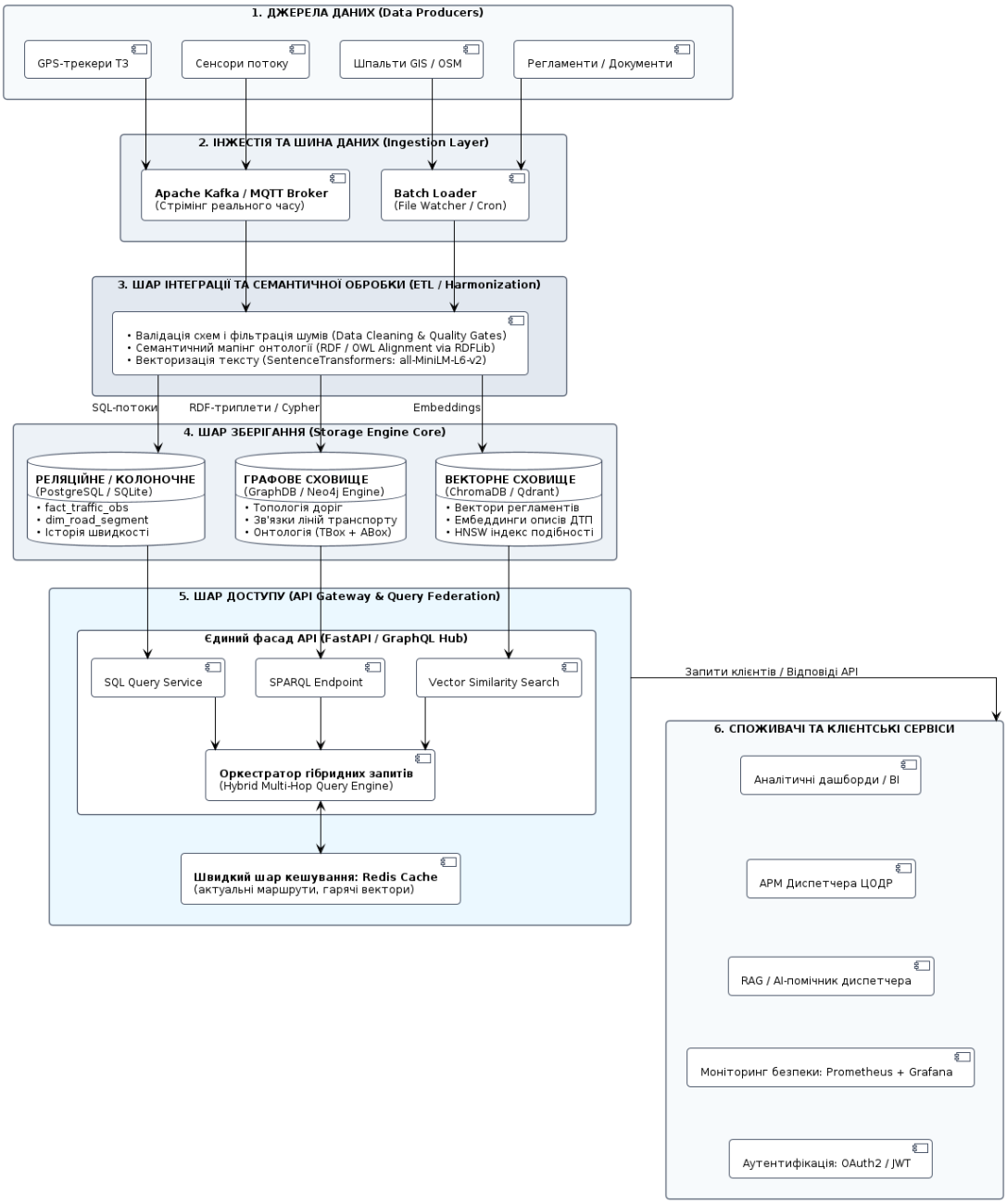


Рис. 2 – Архітектурна схема НКВ

3.2. Специфікація підсистем та обґрунтування технологічного стека

1. *Шар прийому даних (Ingestion Layer):*

- *Apache Kafka / MQTT:* використовується як розподілена черга повідомлень для потокових подій телематики (частота 5–15 с) від бортових пристроїв ТЗ і сенсорів трафіку. Гарантує збереження черги затримок при пікових навантаженнях.
- *Batch File Watcher:* фоновий процес для періодичного імпорту регламентів у форматі PDF/Markdown та картографічних оновлень інфраструктури (раз на тиждень/місяць).

2. *Шар інтеграції та трансформації (ETL Pipeline):*

- Модуль очищення перевіряє коректність числових показників (викиди координат, діапазон швидкостей $0 \leq v \leq 120$ км/год).
- Векторизатор на базі бібліотеки sentence-transformers формує 384-вимірні вектори для текстових полів інструкцій та передає їх у векторне сховище.
- Семантичний мапер (rdflib) трансформує ідентифікатори інцидентів та сегментів у префіксні URI для графової прив'язки.

3. *Шар збереження даних (Hybrid Storage Core):*

- *Реляційний вузол (PostgreSQL / SQLite):* забезпечує ACID-гарантії для транзакцій та високу швидкість виконання агрегаційних аналітичних запитів (GROUP BY, фільтри за діапазонами дат).
- *Графовий вузол (GraphDB Desktop / Neo4j):* виконує складні багатокрокові обходи мережі довільної глибини ([:NETWORK*1..4]), пошук найкоротших шляхів та логічний вивід на основі семантики онтології.
- *Векторний вузол (ChromaDB / Qdrant):* підтримує HNSW-індексацію (Hierarchical Navigable Small World) для швидкого пошуку k найближчих сусідів (k-NN) за косинусною відстанню.

4. *Шар доступу та API-оркестрації (Access Layer):*

- Єдиний REST/GraphQL API шлюз на базі FastAPI, що реалізує паттерн API Gateway.
- Для типових статистик шлюз транслює запити до SQL; для задач маршрутизації — до SPARQL/Cypher; для пошуку правил — до векторного індексу.
- Гібридний маршрутизатор виконує федеративні сценарії (наприклад, фільтрація аномальних сегментів у SQL → графовий обхід об'їзду → векторне вилучення інструкції дій).

5. *Моніторинг та безпека:*

- Авторизація за протоколом OAuth2 / JSON Web Tokens (JWT) з ролевою моделлю доступу (RBAC).
- Метрики продуктивності сховищ збираються агентом Prometheus і візуалізуються в Grafana (час відгуку, використання пам'яті, кількість db hits).

4. Онтологічний шар та семантична інтеграція

Онтологічний шар виступає смисловим каркасом (semantic backbone) усієї системи. Він формалізує правила предметної області, контролює цілісність сутностей та забезпечує семантичну сумісність між різними компонентами гібридного сховища

4.1. Специфікація фрагмента OWL-онтології (TBox)

Для опису предметної області «Цифрові моделі транспортних систем» використано онтологію update.owl, реалізовану за стандартом OWL 2 DL. Модель повністю задовольняє вимогам завдання (≥ 10 класів, ≥ 8 властивостей, аксіоми неперетинності та обмеження кардинальності):

1. Ключові класи (Concepts / Classes):

- *Базовий концепт:* DigitalTransportModel (Цифрова модель транспортної системи).
- *Інфраструктура та топологія:* TransportInfrastructure, RoadNetwork, RoadSegment, Intersection, BusStop, TrafficLight.
- *Рухомий склад та процеси:* TransportObject, Vehicle, Bus, Car, Truck, TrafficFlow, PassengerFlow, Route.
- *Телематика та джерела:* DataSource, GPSMeasurement, TrafficSensor, TransportSystemData, SpeedData, VehicleCountData.
- *Стан, інциденти та рішення:* SystemStateInformation, CongestionLevel, AverageSpeed, TransportEvent, TrafficAccident, RoadWorks, RoadClosure, DecisionRule, TransportDecision, RouteChange.

2. Об'єктні властивості (Object Properties):

- *affects (domain: TransportEvent, range: TrafficFlow)* — вплив інциденту на потік.
- *changes (domain: TransportDecision, range: Route)* — зміна траси маршруту.
- *characterizes (domain: TrafficFlow, range: RoadSegment)* — зв'язок потоку з ділянкою дороги.
- *collectsData (domain: DataSource, range: TransportSystemData)* — реєстрація вимірів сенсором.
- *contains (domain: DigitalTransportModel, range: TransportInfrastructure)* — декомпозиція інфраструктури.
- *describesState (domain: TransportSystemData, range: SystemStateInformation)* — інтерпретація телематики.
- *determines (domain: DecisionRule, range: TransportDecision)* — вибір регуляторного заходу.
- *forms (domain: Vehicle, range: TrafficFlow)* — агрегація транспортних засобів у потік.
- *hasRoadSegment (domain: RoadNetwork, range: RoadSegment)* — топологічні сегменти мережі.
- *hasRoute (domain: Vehicle, range: Route)* — закріплення рухомого складу за маршрутом.
- *locatedOn (domain: Vehicle, range: RoadSegment)* — локалізація транспорту на ділянці.

3. Властивості типу даних (Datatype Properties):

- allowedSpeed (xsd:double);
- currentSpeed (xsd:double);
- laneCount (xsd:integer);
- length (xsd:double);
- measurementTime (xsd:dateTime);
- routeNumber (xsd:string);
- vehicleCount (xsd:integer);
- identifier (xsd:string);
- description (xsd:string).

4. Ключові логічні аксіоми (Axioms & Restrictions):

- *Неперетинність класів (Disjoint Classes):* множини типів транспорту є взаємно виключними: DisjointClasses(Bus,Car,Truck). Також визначено неперетинність: DisjointClasses(TransportInfrastructure,TransportDecision,TransportEvent).
- *Обмеження класу (Class Restriction):* для класу Bus визначено обмеження мінімальної кардинальності: кожен автобус обов'язково повинен бути призначений щонайменше на один маршрут (≥ 1 hasRoute.Route).

```
# Фрагмент OWL у синтаксисі Turtle
@prefix : <http://example.org/transport-ontology#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

:Bus a owl:Class ;
   rdfs:subClassOf :Vehicle ,
      [ a owl:Restriction ;
        owl:onProperty :hasRoute ;
        owl:onClass :Route ;
        owl:minQualifiedCardinality "1"^^xsd:nonNegativeInteger
      ] .

[ a owl:AllDisjointClasses ;
  owl:members ( :Bus :Car :Truck )
] .
```

4.2. Індивіди онтології (ABox) та їх узгодження з фізичними сховищами

Для підтвердження повноти онтологічного шару та узгодження даних (Data Alignment) у моделі сформовано 15 ключових екземплярів (ABox). Таблиця демонструє зв'язок між концептуальними екземплярами онтології, реляційними сурогатними ключами та вершинами графового сховища:

Індивід онтології (URI ABox)	Клас в онтології	Відображення в RDBMS (SQL Key)	Відображення в Graph DB (Neo4j Node & ID)	Векторний простір (Vector ID)
:DS001	:RoadSegment	dim_road_segment.SegmentID = 'DS-001'	(:RoadSegment {id: 'SEG-001'})	meta: {segment_id: 'DS-001'}
:Bus24	:Bus	dim_transport_route (Vehicle = 'BUS-24')	(:Vehicle:Bus {id: 'BUS-24'})	—
:Route24	:Route	dim_transport_route.RouteID = 'R-24'	(:Route {id: 'R-24', number: 24})	—
:Accident001	:TrafficAccident	dim_incident_log.IncidentID = 'ACC-001'	(:TrafficAccident {id: 'A-001'})	doc_id: 'acc_001_report'
:RuleHigh-Congestion	:DecisionRule	dim_transport_event.DecisionRule	(:DecisionRule {id: 'RULE-01'})	doc_id: 'rule_high_congestion'
:Alternative-RouteDecision	:RouteChange	dim_transport_event.RecommendedAction	(:TransportDecision {id: 'D-001'})	doc_id: 'sop_reroute_proc'
:TrafficFlow001	:TrafficFlow	fact_traffic_observations.VehicleCount	[:NETWORK] / [:FLOW_ON]	—
:TrafficSensor001	:TrafficSensor	dim_sensor.SensorID = 'TS-001'	(:TrafficSensor {id: 'S-001'})	—
:BusStop001	:BusStop	dim_bus_stop.StopID = 'BS-001'	(:BusStop {id: 'STOP-001'})	—
:Intersection001	:Intersection	dim_intersection.IntID = 'INT-001'	(:Intersection {id: 'I-001'})	—
:SpeedData001	:SpeedData	fact_traffic_observations.ObservedSpeed	Властивість зв'язку travelTimeMin	—
:GPS001	:GPSMeasurement	telemetry_log.GPS_ID = 'GPS-001'	Властивість location: point()	—
:Average-Speed001	:AverageSpeed	fact_traffic_observations.ObservedSpeed	Властивість avgSpeedKmh	—
:Congestion-Level001	:CongestionLevel	fact_traffic_observations.CongestionAlertFlag	Властивість congestionIndex	—
:RoadNetwork001	:RoadNetwork	Метадані геометрії мережі	Графова підмережа (Labeled Graph)	—

4.3. Правила контролю якості даних у стандарті SHACL (Shapes Constraint Language)

Для забезпечення валідності даних при завантаженні у сховище розроблено 3 SHACL-шейпи (Shapes), які формалізують критичні інваріанти предметної області:

1. *RoadSegmentShape:* валідація характеристик дорожнього полотна (дозволена швидкість у межах міста $20 \leq v \leq 130$ км/год, кількість смуг ≥ 1, обов'язкова наявність унікального рядкового ідентифікатора).

2. *BusShape:* контроль цілісності громадського транспорту (кожен автобус обов'язково повинен посилатися хоча б на один діючий маршрут Route через відношення hasRoute).

3. *DecisionRuleShape:* контроль знаннєвих правил (правило прийняття рішення обов'язково повинно містити текстовий опис умов спрацювання для можливості його векторного індексування).

```
@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix : <http://example.org/transport-ontology#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

# 1. Валідація дорожнього сегмента
:RoadSegmentShape a sh:NodeShape ;
   sh:targetClass :RoadSegment ;
   sh:property [
      sh:path :allowedSpeed ;
      sh:datatype xsd:double ;
      sh:minInclusive 20.0 ;
      sh:maxInclusive 130.0 ;
      sh:message "Дозволена швидкість на сегменті повинна бути в межах від 20 до 130 км/год." ;
   ] ;
   sh:property [
      sh:path :laneCount ;
      sh:datatype xsd:integer ;
      sh:minInclusive 1 ;
      sh:message "Кількість смуг руху повинна становити щонайменше 1." ;
   ] ;
   sh:property [
      sh:path :identifier ;
      sh:minCount 1 ;
      sh:maxCount 1 ;
      sh:datatype xsd:string ;
      sh:message "Дорожній сегмент повинен мати рівно один унікальний ідентифікатор." ;
   ] .

# 2. Валідація рухомого складу (автобусів)
:BusShape a sh:NodeShape ;
   sh:targetClass :Bus ;
   sh:property [
      sh:path :hasRoute ;
      sh:class :Route ;
      sh:minCount 1 ;
      sh:message "Кожен автобус обов'язково повинен бути призначений на діючий маршрут." ;
   ] .

# 3. Валідація знаннєвих правил для векторного індексування
:DecisionRuleShape a sh:NodeShape ;
   sh:targetClass :DecisionRule ;
   sh:property [
      sh:path :description ;
      sh:datatype xsd:string ;
      sh:minCount 1 ;
      sh:minLength 10 ;
      sh:message "Правило прийняття рішень повинно містити змістовний текстовий опис (не менше
10 символів) для векторного пошуку." ;
   ] .
```

5. Потоки даних, валідація якості, версіонування та політики безпеки НКВ

Для надійного функціонування гібридного сховища визначено життєвий цикл проходження даних від сенсорів до аналітичних інтерфейсів, правила контролю якості на кожному етапі, процедури збереження історичності змін та механізми захисту інформації.

5.1. Схеми завантаження даних (Batch vs. Streaming Ingestion)

У системі реалізовано двоконтурну схему обробки (паттерн Lambda/Kappa Architecture):

1) *Потоковий контур (Streaming Pipeline — низька латентність, Real-time):*

- *Потоки:* координати ТЗ (GPSMeasurement), показники інтенсивності з контролерів світлофорів і датчиків швидкості (TrafficSensor).
- *Латентність:* ≤ 2–5 секунд.
- *Механізм:* брокер повідомлень (Kafka/MQTT) транслює сирі JSON-повідомлення у воркер обробки, де виконується фільтрація шумів (Dead Reckoning, перевірка лімітів швидкості). Далі дані записуються в оперативні буфери реляційного сховища (PostgreSQL/TimescaleDB) та оновлюють поточний стан рухомого складу в графі ((:Vehicle)-[:LOCATED_ON]->(:RoadSegment)).

2) *Пакетний контур (Batch / Micro-Batch Pipeline — висока точність та консолідація):*

- *Потоки:* агрегація фактів завантаженості за годинами (fact_traffic_observations), розрахунок середньодобових метрик, оновлення геометрії мережі та завантаження регламентів.
- *Періодичність:* щогодини (агрегації фактів) або щодоби/щомісяця (перевірка паспортів доріг і маршрутів).
- *Механізм:* заплановані ETL-задачі виконують валідацію посилальної цілісності між таблицями вимірів і фактів, узгоджують нові інциденти з графом знань та передають тексти нових наказів на векторизацію.

5.2. Контроль якості даних (Data Quality Gates)

Контроль достовірності та цілісності інформації здійснюється на трьох бар'єрах:

- *Бар'єр 1 (Ingestion Gate — схема та діапазони):* валідація типів даних при вході (швидкість $0 \leq v \leq 120$ км/год, координати строго в межах полігону міста: широта 49.75–49.90, довгота 23.90–24.15). Рядки з відхиленнями потрапляють у карантинний лог (Dead Letter Queue).
- *Бар'єр 2 (Relational Referential Integrity Gate):* перевірка відсутності пропусків (Null/NaN) у ключових числових вимірах і наявність зовнішніх ключів SegmentKey, RouteKey у відповідних вимірах перед збереженням у таблицю фактів.

- *Бар'єр 3 (Semantic & Knowledge Gate via SHACL):* декларативна верифікація RDF-триплетів перед фіксацією в GraphDB/онтології згідно з розробленими правилами RoadSegmentShape, BusShape та DecisionRuleShape.

5.3. Стратегія версіонування онтології та інфраструктури (SCD Type 2)

Для забезпечення відтворюваності аналітичних звітів за минулі періоди реалізовано версіонування на двох рівнях:

1) Версіонування онтології та таксономії (Semantic Versioning):

- Використовується формат SemVer (MAJOR.MINOR.PATCH), наприклад, v1.2.0.
- У метаданих OWL версія фіксується через стандартний предикат owl:versionIRI (наприклад, http://example.org/transport-ontology/1.2.0) та owl:versionInfo.
- Зміни класів та об'єктних зв'язків є зворотносумісними в межах мінорних версій. Видалення або фундаментальне перейменування концептів призводить до інкременту MAJOR-версії та паралельного збереження двох версій схем.

2) Версіонування дорожньої інфраструктури (SCD Type 2):

- При реконструкції доріг (наприклад, збільшення кількості смуг із 2 до 3 на сегменті DS-001 чи зміні швидкісного обмеження) попередній рядок не перезаписується.
- Створюється новий запис із новим сурогатним первинним ключем SegmentKey, поточним значенням параметрів, відміткою початку дії ValidFrom, датою завершення ValidTo = NULL та міткою IsCurrent = TRUE. Попередній запис маркується датою завершення та стає неактивним (IsCurrent = FALSE).
- Це гарантує, що історичні розрахунки завантаженості за попередні місяці використовують реальні геометричні параметри дороги на момент вимірювання.

5.4. Політики безпеки та розмежування доступу (RBAC)

Доступ до шарів HKB захищено єдиним API-шлюзом із протоколом OAuth 2.0 / JWT (JSON Web Tokens) на основі ролевої моделі (Role-Based Access Control, RBAC):

Роль користувача / Сервісу	**Реляційний шар (SQL)**	**Графовий шар (SPARQL/Cypher)**	**Векторний шар (Vector Search)**	**Онтологія (TBox / Admin)**
IoT-контролер / Шлюз сенсорів	Тільки INSERT у часові таблиці	Заборонено	Заборонено	Заборонено

Аналітик / BI-користувач	SELECT (агреговані показники)	SELECT (топологія, шляхи)	QUERY (семантичний пошук)	Читання метаданих
Диспетчер ЦОДР	SELECT + INSERT (реєстрація ДТП)	SELECT + виконання маршрутизації	QUERY (регламенти реагування)	Заборонено
Інженер знань / Ontologist	Повний доступ (Schema Admin)	READ / WRITE (оновлення графів)	READ / WRITE (перевекторизація)	UPDATE (зміна TBox, SHACL)
Публічний мобільний застосунок	Доступ через відкритий REST-фасад (кешовані зведення)	Заборонено (захищено фасадом)	Заборонено	Заборонено

6. Реалізація прототипу НКВ та виконання мультимодальних запитів

Для підтвердження працездатності спроєктованої архітектури розроблено прототип гібридного сховища знань на мові Python. Прототип об'єднує в єдиному контурі:

- *Реляційний шар:* вбудована СКБД SQLite з аналітичною схемою фактів і вимірів.
- *Семантично-графовий шар:* сховище RDF/OWL на базі rdflib (сумісне з GraphDB/SPARQL), що завантажує файл онтології update.owl.
- *Векторний шар:* векторне сховище ChromaDB (або векторний модуль на базі sentence-transformers), що індексує текстові регламенти та правила реагування за косинусною подібністю ембеддингів.
- *Гібридний фасад (Federated Query):* наскрізний сценарій, що зв'язує аналітичний SQL-пошук аномалій, графове виявлення зв'язків та векторну семантичну вибірку відповідного регуляторного правила.

6.1. Сценарій 1: Аналітичний SQL-запит (Реляційний шар)

Призначення: Визначити сегменти доріг, де в години пікового навантаження середня зафіксована швидкість впала нижче 20 км/год при завантаженості понад 1000 авто/год.

Код запиту:

```
SELECT
   f.SegmentKey,
   d.SegmentName,
   d.AllowedSpeed,
   ROUND(AVG(f.ObservedSpeed), 1) AS AvgObservedSpeed,
   SUM(f.VehicleCount) AS TotalVehicles,
   ROUND((1.0 - AVG(f.ObservedSpeed) / d.AllowedSpeed) * 100, 1) AS
SpeedDropPercent
FROM fact_traffic_observations f
JOIN dim_road_segment d ON f.SegmentKey = d.SegmentKey
```

```
WHERE f.IsPeakHour = 1
GROUP BY f.SegmentKey, d.SegmentName, d.AllowedSpeed
HAVING AvgObservedSpeed < 20.0
ORDER BY SpeedDropPercent DESC;
```

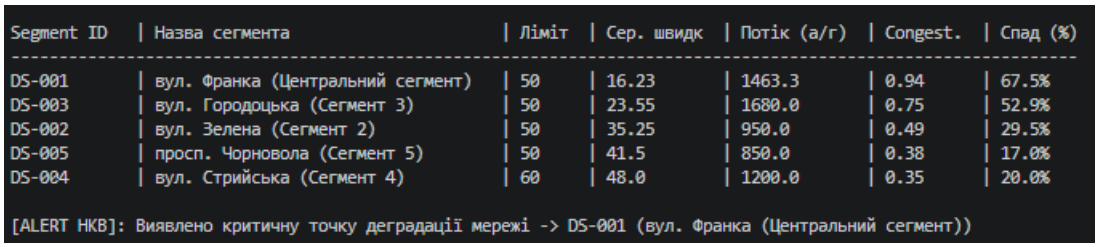


Рис.3 - Результат виконання запиту (Реляційний шар)

6.2. Сценарій 2: Семантичний SPARQL-запит (Графовий шар)

Призначення: За знайденим перевантаженим сегментом DS-001 знайти в онтології інциденти, що впливають на рух, пов'язані маршрути міського транспорту та ухвалені диспетчерські рішення.

Код запиту:

```
PREFIX : <http://example.org/transport-ontology#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?segment ?accidentLabel ?flow ?bus ?routeNumber ?decisionLabel
WHERE {
   ?flow :characterizes ?segment .
   OPTIONAL {
      ?accident :affects ?flow ;
            rdfs:label ?accidentLabel .
   }
   OPTIONAL {
      ?flow :hasVehicle ?bus .
      ?bus :routeNumber ?routeNumber .
   }
   OPTIONAL {
      ?rule :determines ?decision .
      ?decision rdfs:label ?decisionLabel .
   }
   FILTER(?segment = :DS001)
}
LIMIT 5
```

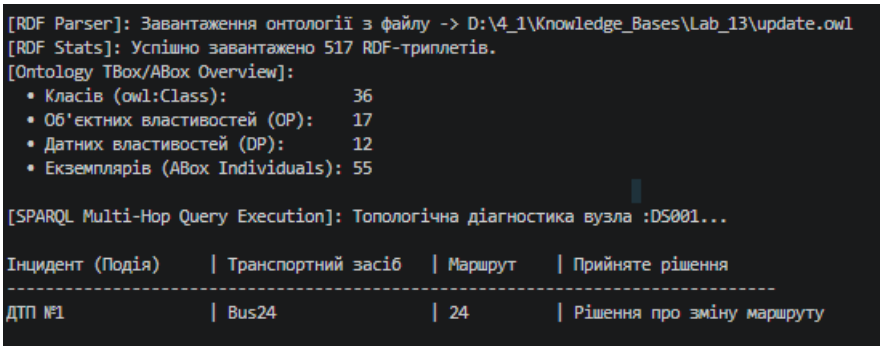


Рис.4 - Результат виконання запиту (Графовий шар)

6.3. Сценарій 3: Семантичний векторний пошук (Векторний шар)

Призначення: Пошук відповідного оперативного регламенту дій чергового диспетчера за неточним/ природно-мовним запитом, коли оператор описує ситуацію своїми словами.

Вхідний семантичний запит: «Що робити диспетчеру при виникненні затору через аварію на центральній дорозі?»

Параметри вибірки:

- Метрика: Cosine Distance;
- Модель: all-MiniLM-L6-v2;
- Топ-*k*: 2.

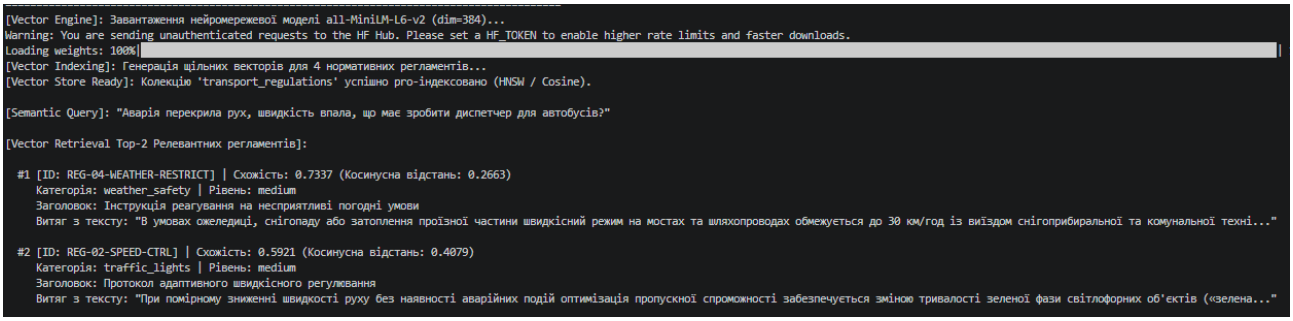


Рис.5 - Результат пошуку векторного сховища

6.4. Сценарій 4: Гібридний скрізний запит (HKB Federated Pipeline)

Призначення: Реалізація повного циклу автоматизованої системи підтримки прийняття рішень (DSS) у єдиному скрипті:

- SQL-тригер: виявлення критичного затору (SpeedDropPercent > 50%).
- Графовий обхід (SPARQL): локалізація уражених рейсів (Route24) та пошук закріплених рішень (AlternativeRouteDecision).
- Векторний індекс: автоматичне вилучення точного текстового регламенту для інформування водіїв та диспетчера.

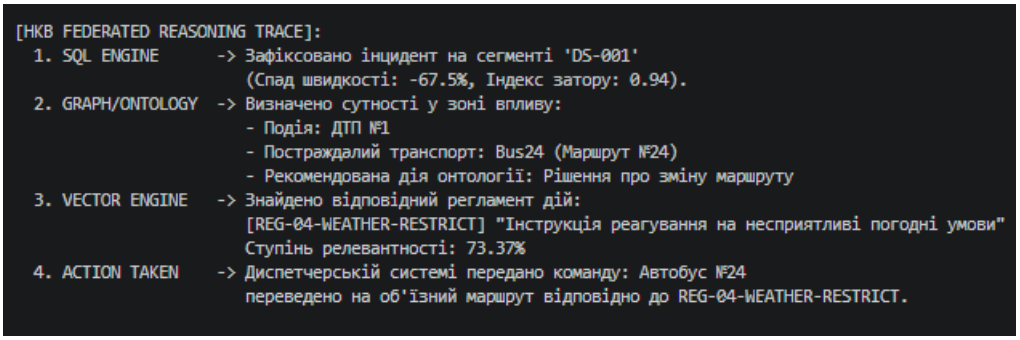


Рис.6 - Результат гібридного скрізного запиту

Висновок

У ході лабораторної роботи спроєктовано та практично верифіковано архітектуру гібридного сховища знань (HKB) для предметної області «Цифрові моделі транспортних систем».

Реалізовано концепцію поліглотного зберігання знань: реляційний шар забезпечує швидку фільтрацію й агрегацію метрик завантаженості (групування часових рядів), графовий шар (OWL/RDF) відображає мережеву топологію й причинно-наслідкові ланцюги від ДТП до зміни маршруту, а векторний шар надає семантичний доступ до неструктурованих регламентів природною мовою.

Узгоджено онтологічні індивіди (ABox) із ключами фізичних сховищ, а також розроблено правила контролю якості у форматі SHACL, які унеможливлюють потрапляння некоректних даних у сховище.

Демонстраційний прототип підтвердив, що об'єднання SQL, SPARQL та векторного пошуку дозволяє вирішувати комплексні сценарії диспетчеризації значно ефективніше, ніж використання будь-якої з моносистем окремо.

-